TRIỂN KHAI MÔ HÌNH SVM IRIS

Ứng dụng phân loại hoa Iris với 4 Kernel
Linear • RBF • Polynomial • Sigmoid

Sinh viên thực hiện: Lê Thanh Thảo



1. Mục tiêu và phạm vi dự án

Mục tiêu 1

Xây dựng và huấn luyện 4 mô hình SVM trên bộ dữ liệu Iris.

Mục tiêu 2

Đánh giá Accuracy, Precision, Recall, F1 và thời gian thực thi.

Mục tiêu 3

Đóng gói thành API và xây dựng giao diện web để người dùng thử nghiệm.

Phạm vi chính

- Dữ liệu chuẩn Iris: 150 mẫu, 4 đặc trưng, 3 lớp.
- Chia dữ liệu 80% Train / 20% Test, random_state = 42, stratify = True.
- Huấn luyện riêng Linear, RBF, Polynomial và Sigmoid.
- Lưu mô hình thành 4 file .pkl và lưu kết quả vào metrics.json / weights.json.
- Cung cấp các chức năng dự đoán, trực quan hóa, benchmark và phân tích dữ liệu trên web.

2. Kiến trúc tổng thể hệ thống



Người dùng → nhập thông số → chọn/điều khiển mô hình → nhận kết quả phân loại

Backend Python

app.py tải các file svm_linear.pkl, svm_rbf.pkl, svm_poly.pkl, svm_sigmoid.pkl và phục vụ endpoint /predict.

Frontend / Server Node

server.ts cung cấp web production, endpoint phụ trợ và đọc metrics.json / weights.json.

3. Cấu trúc thư mục dự án


```
Iris-SVM-Update/
├── train.py
├── app.py
├── server.ts
├── metrics.json
├── weights.json
├── svm_linear.pkl
├── svm_rbf.pkl
├── svm_poly.pkl
├── svm_sigmoid.pkl
├── render.yaml
├── package.json
├── index.html
└── src/
    ├── components/
    ├── utils/
    ├── data/
    └── types/
```

train.py

Huấn luyện 4 kernel, tính metrics, lưu model và kết quả.

app.py

FastAPI: nạp model .pkl, nhận input và trả kết quả dự đoán.

src/

Giao diện React/TypeScript: dự đoán, benchmark, experiments, visualization, file analysis.

render.yaml + package.json

Cấu hình build/start và triển khai Web Service trên Render.

4. Bộ dữ liệu Iris



Iris Setosa



Iris Versicolor



Iris Virginica

150 mẫu

50 mẫu cho mỗi loài.

4 đặc trưng

Sepal length/width; Petal length/width.

3 lớp

Setosa, Versicolor, Virginica.

Đơn vị

Các đặc trưng được đo bằng cm.

Chia dữ liệu: 120 mẫu Train + 30 mẫu Test; stratify giúp duy trì tỷ lệ các lớp.

5. Bốn Kernel được sử dụng

Linear

$K(x,z)=x \cdot z$

Ranh giới tuyến tính; cấu hình C=1.0.

RBF

$\exp(-\gamma||x-z||^2)$

Phi tuyến Gaussian; C=1.0, gamma='scale'.

Polynomial

$(\gamma x \cdot z + 1)^d$

Đa thức bậc 3; C=1.0, degree=3, coef0=1.0.

Sigmoid

$\tanh(\gamma x \cdot z)$

Kernel dạng hyperbolic; C=1.0, gamma='scale'.

Điểm quan trọng: mỗi kernel được huấn luyện thành một model riêng và lưu thành một file .pkl.

6. Bước 1 — Chuẩn bị và chia dữ liệu

1 **Tải Iris**
datasets.load_iris() lấy X và y từ bộ dữ liệu chuẩn.

2 **Tách Train/Test**
80% dùng huấn luyện, 20% dùng đánh giá; random_state=42.

3 **Stratify**
Giữ cân bằng tỷ lệ 3 lớp trong tập Train và Test.

4 **Xác định kernel**
Lần lượt chạy linear → rbf → poly → sigmoid.

```
iris = datasets.load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test =
    train_test_split(
        X, y,
        test_size=0.2,
        random_state=42,
         stratify=y
)
```

Kết quả chia dữ liệu

150 tổng → 120 Train + 30 Test. Mỗi lớp có 10 mẫu trong tập Test.

7. Bước 2 — Huấn luyện 4 mô hình

train.py

```
kernels = ["linear", "rbf", "poly", "sigmoid"]

for kernel in kernels:
    if kernel == "poly":
        model = SVC(kernel="poly",
                    degree=3, C=1.0,
                    coef0=1.0, gamma="scale")
    elif kernel == "rbf":
        model = SVC(kernel="rbf",
                    C=1.0, gamma="scale")
    elif kernel == "sigmoid":
        model = SVC(kernel="sigmoid",
                    C=1.0, gamma="scale")
    else:
        model = SVC(kernel="linear", C=1.0)

    model.fit(X_train, y_train)
    joblib.dump(model, f"svm_{kernel}.pkl")
```

Bước 2.1

Khởi tạo SVC theo từng kernel.

Bước 2.2

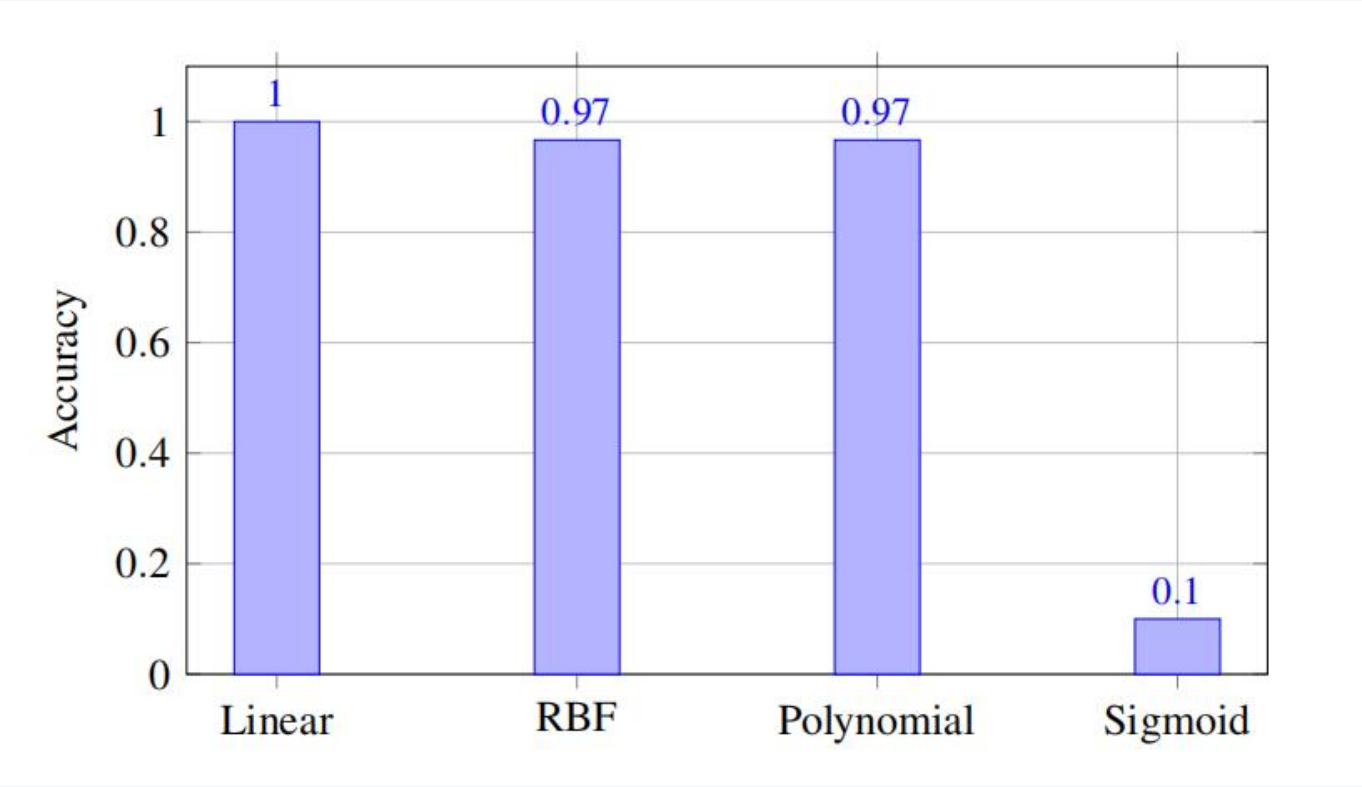
Fit model trên 120 mẫu Train.

Bước 2.3

Lưu 4 model độc lập bằng joblib.

Output: svm_linear.pkl • svm_rbf.pkl • svm_poly.pkl • svm_sigmoid.pkl

8. Bước 3 — Đánh giá mô hình



Accuracy

Tỷ lệ dự đoán đúng trên 30 mẫu Test.

Precision / Recall

Đánh giá chất lượng dự đoán theo từng lớp; báo cáo dùng macro average.

F1-score

Trung bình điều hòa giữa Precision và Recall.

Thời gian

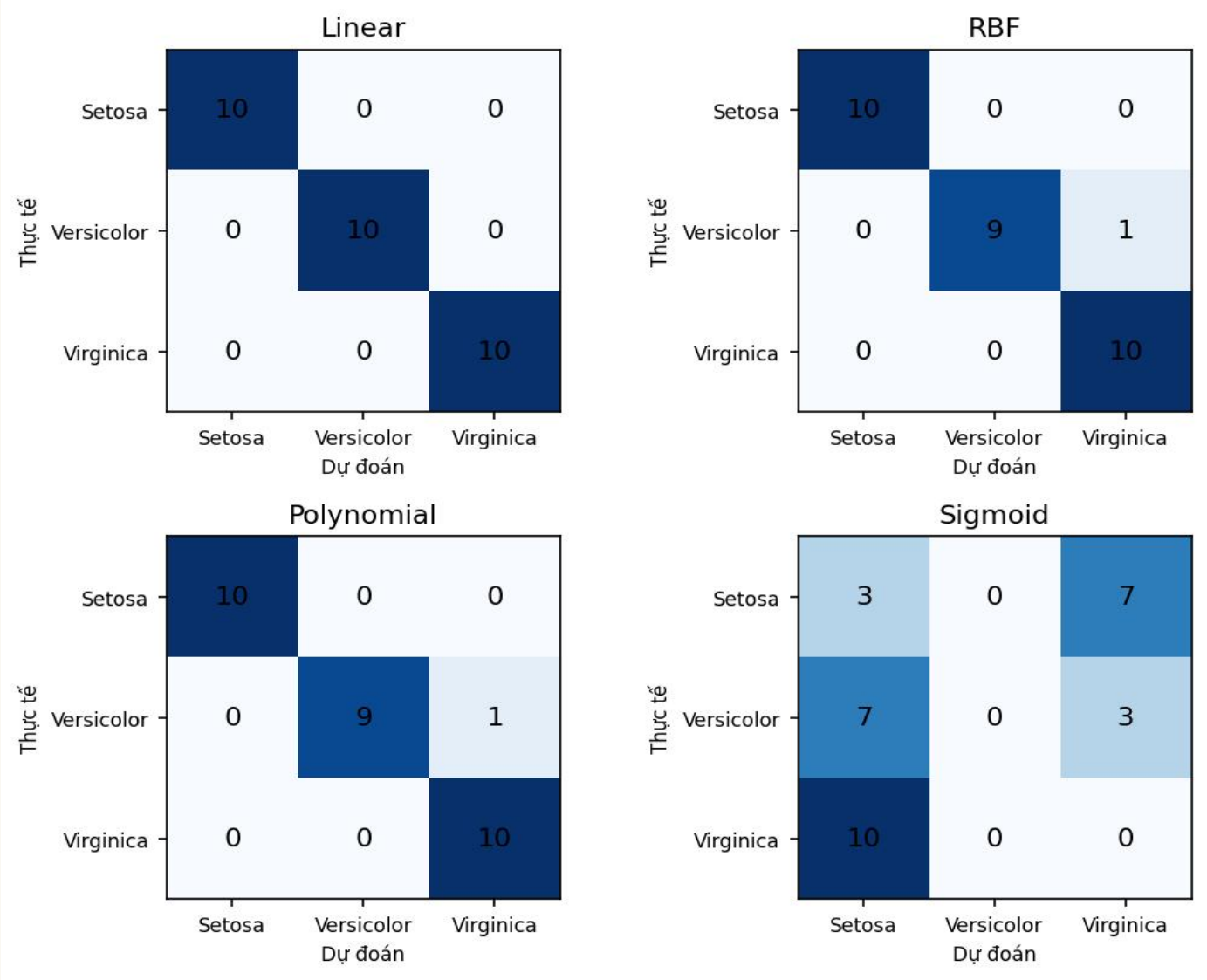
Đo riêng train_time_ms và pred_time_ms cho từng kernel.

9. Kết quả thực nghiệm từ metrics.json

Kernel	Accuracy	Precision	Recall	F1	Train (ms)	Predict (ms)
Linear	100.00%	100.00%	100.00%	100.00%	1.36	0.22
RBF	96.67%	96.97%	96.67%	96.66%	1.13	0.22
Polynomial	96.67%	96.97%	96.67%	96.66%	0.89	0.14
Sigmoid	10.00%	5.00%	10.00%	6.67%	1.68	0.30

Các số liệu trên được đọc trực tiếp từ metrics.json của dự án; không phải số liệu minh họa.

10. Ma trận nhầm lẫn (Confusion Matrix)



Linear

Ma trận đường chéo hoàn toàn: 30/30 mẫu được phân loại đúng.

RBF & Polynomial

Mỗi mô hình nhầm 1 mẫu Versicolor thành Virginica → 29/30 đúng.

Sigmoid

Chỉ 3/30 Setosa đúng; nhiều mẫu bị dồn sang Setosa/Virginica.

11. Bước 4 — Lưu mô hình và kết quả

1. **Lưu model**
   joblib.dump(model, 'svm_<kernel>.pkl').

2. **Lưu metrics**
   Accuracy, Precision, Recall, F1, support vectors và confusion matrix.

3. **Lưu cấu hình**
   weights.json lưu params, dataset info, thời gian và thông tin các model.

4. **Sẵn sàng deploy**
   Backend có thể nạp lại các model đã huấn luyện mà không cần train lại.

```
joblib.dump(model, f"svm_{kernel}.pkl")

json.dump(metrics_results, f,
          indent=4, ensure_ascii=False)
```

Thông tin kèm theo

150 mẫu • 4 features • 3 classes • test_size=0.2 • random_state=42 • stratify=True

Thời điểm trong weights.json

trained_at: 2026-09-26 14:20:44

12. Bước 5 — Xây dựng FastAPI

```
models = {}
for k in ["linear", "rbf", "poly", "sigmoid"]:
    models[k] = joblib.load(f"svm_{k}.pkl")
```

GET /health

Kiểm tra trạng thái server và số model đã được load.

GET /metrics

Trả về metrics đã được tính từ quá trình train.

POST /predict

Nhận 4 số đo + kernel, trả về class_id, tên loài và thời gian thực thi.

Pydantic BaseModel kiểm tra dữ liệu đầu vào; Literal giới hạn kernel hợp lệ thành linear / rbf / poly / sigmoid.

CORS được bật để frontend có thể gọi API từ trình duyệt.

13. Bước 6 — Luồng dự đoán qua API


```python
class IrisInput(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float
    kernel: Literal[
        "linear", "rbf", "poly", "sigmoid"
    ] = "linear"

@app.post("/predict")
def predict(data: IrisInput):
    k = data.kernel.lower()
    selected_model = models[k]
    features = np.array([[
        data.sepal_length,
        data.sepal_width,
        data.petal_length,
        data.petal_width
    ]])
    prediction = int(selected_model.predict(features)[0])
    return {
        "kernel_used": k,
        "class_id": prediction,
        "prediction": SPECIES_MAP[prediction]
    }
```

Bước 6.1 — Nhận input

4 đặc trưng + kernel được gửi dưới dạng JSON.

Bước 6.2 — Chọn model

models[k] lấy đúng file .pkl tương ứng với kernel.

Bước 6.3 — Trả kết quả

class_id + prediction + kernel_used + execution_time_ms.

14. Bước 7 — Giao diện Web

Nhận diện

Nhập 4 số đo bằng slider hoặc ô số; gửi yêu cầu phân loại.

Visualization

Trực quan 2D 150 mẫu và chọn cặp đặc trưng X/Y.

Bạn đoán trước

Đoán loài hoa trước khi mô hình đưa ra dự đoán.

Phân tích dữ liệu

Tải/đọc dữ liệu và phân tích các đặc trưng.

Benchmark

So sánh Accuracy, F1, support vectors và thời gian của 4 kernel.

Lịch sử nhận diện

So sánh Accuracy, F1, support vectors và thời gian của 4 kernel.

15. Bước 8 — GitHub và Render

1. **Chuẩn bị production**
   npm install → npm run build → tạo thư mục dist.

2. **GitHub**
   Commit toàn bộ mã nguồn, model .pkl, metrics và cấu hình.

3. **Render**
   Kết nối repository và tạo Web Service runtime Node.

4. **Start server**
   npm start → tsx server.ts; server lắng nghe process.env.PORT.

```
services:
  - type: web
    name: iris-svm
    runtime: node
    buildCommand: npm install && npm run build
    startCommand: npm start
    healthCheckPath: /health
```

Health Check

Render kiểm tra /health để xác nhận dịch vụ đang hoạt động.

Auto Deploy

render.yaml đặt autoDeployTrigger: commit.

16. Quy trình Demo — từ input đến kết quả

1 **Nhập dữ liệu**
Ví dụ: 5.1 / 3.5 / 1.4 / 0.2

2 **Chọn kernel**
Linear, RBF, Poly hoặc Sigmoid

3 **Gửi dự đoán**
Frontend gọi POST /predict

4 **Model xử lý**
Model tương ứng được chọn và predict

5 **Hiển thị**
Tên loài + thông tin đầu vào + thời gian

6 **Đối chiếu**
Benchmark / confusion matrix / metrics

```
POST /predict
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2,
  "kernel": "linear"
}
```

Kết quả theo metrics: mẫu này thuộc Setosa khi dự đoán bằng model Linear.